

Programlamaya Giriş

HAFTA 2

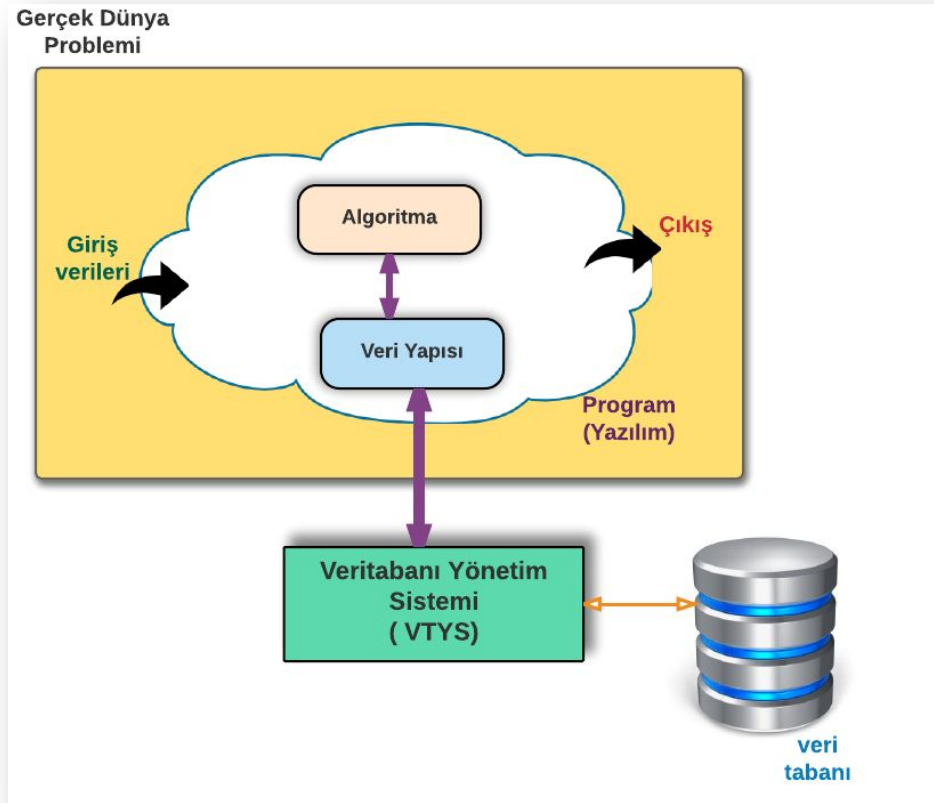
Yazılım Geliştirme Yaşam Döngüsü, Algoritma ve Akış Diyagramları

Konu & İçerik

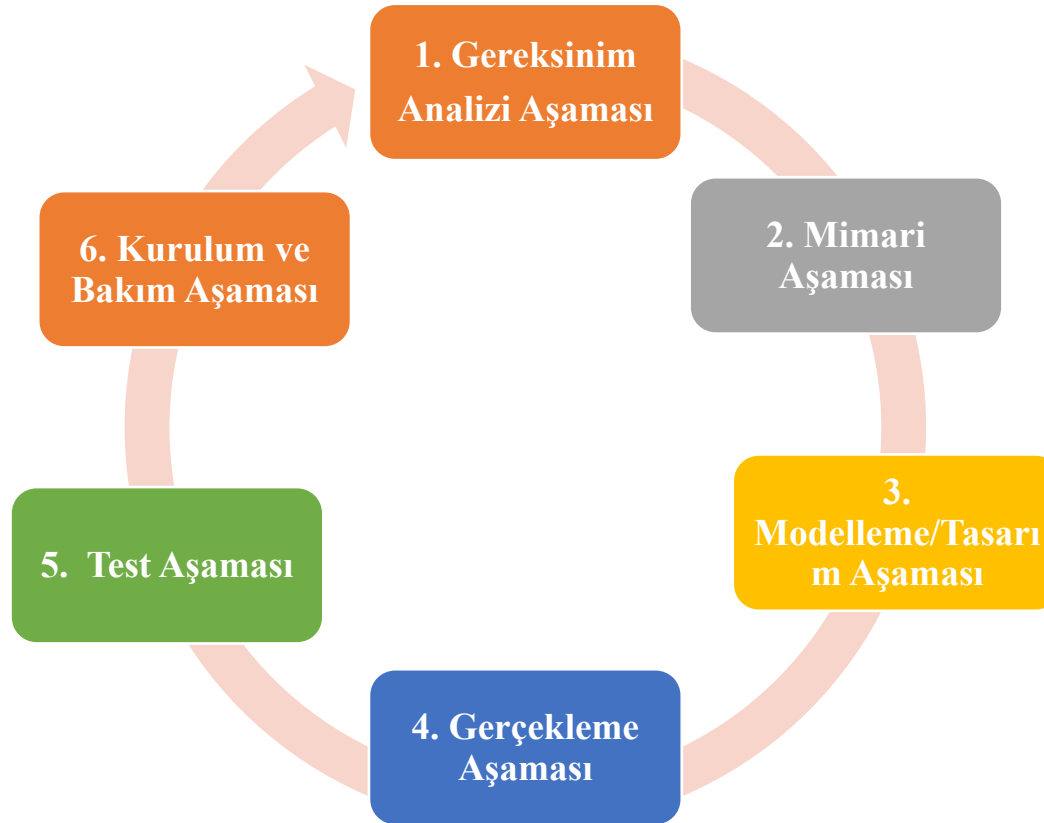
- Büyük Resim – Algoritmik Problem
- Yazılım Geliştirme Yaşam Döngüsü
- ATM Sistemi İçin Yazılım Geliştirme
 - Gereksinim Analizi
 - Modelleme & Tasarım Aşaması
 - UML Dili
 - Kullanım Senaryosu (Use Case Diagram)
 - Etkinlik Şeması (Activity Diagram)
- Sözde Kod ve Akış Şeması

Büyük Resim – Algoritmik Problem

- Gerçek dünya problemlerini bilgisayar yardımıyla çözmek için **yazılım (program)** geliştirilir



Yazılım Geliştirme Yaşam Döngüsü



ATM Sistemi için Yazılım Geliştirme

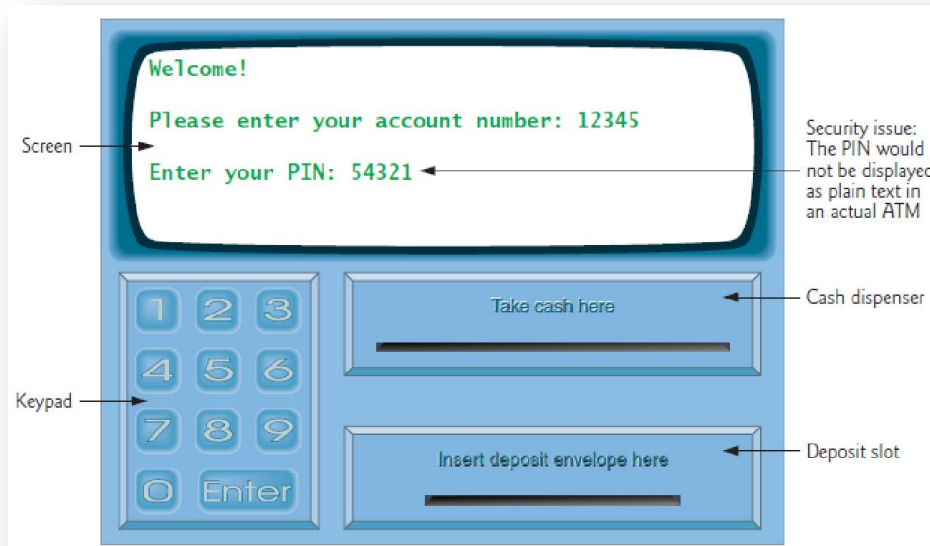
- Yerel bir banka, kullanıcılarının (yani, banka müşterilerine) temel finansal işlemlerini gerçekleştirebilmelerine izin veren yeni bir ATM yazılımı yüklemek istiyor.
- Her kullanıcının bankada sadece bir hesabı olabilir.
- ATM, kullanıcıların hesap bakiyelerini görmelerine, hesaplarından para çekmelerine ve hesaplarına para yatırmalarına izin vermelidir.

ATM Sistemi için Yazılım Geliştirme...

■ Gereksinim Analizi Aşaması

■ **ATM** makinasının kullanıcı arayüzü aşağıdaki donanım bileşenlerinden oluşmalıdır:

- **Tuş takımı** kullanıcının girdiği bilgiyi okur
- **Ekran** kullanıcıya mesaj görüntüler
- **Para yatırma bölümü** kullanıcından para zarfını alır
- **Para verme bölümü** kullanıcıya para verir

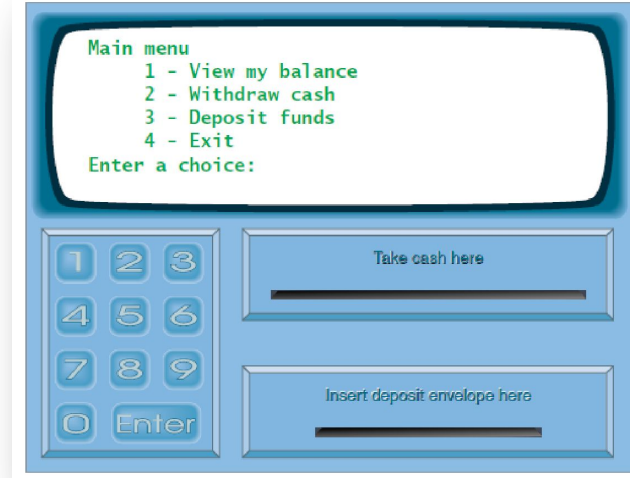


ATM Sistemi için Yazılım Geliştirme...

■ Gereksinim Analizi Aşaması...

□ ATM ye ilk girişte (şu anda hiç kimsenin kullanmadığı varsayılarak), kullanıcı aşağıdaki işlem adımlarını gerçekleştirir.

1. Ekran bir «Hoşgeldiniz!» mesajı görüntüler ve kullanıcıdan bir hesap numarası girmesini ister.
2. Kullanıcı tuş takımını kullanarak beş basamaklı hesap numarasını girer
3. Ekran, kullanıcıdan ilgili hesap numarası ile ilişkili PIN'i girmesini ister
4. Kullanıcı, tuş takımını kullanarak beş basamaklı PIN'i girer.
5. Eğer kullanıcı geçerli bir hesap numarası ve bu hesap için doğru PIN girdi ise ekranda ana menü görüntülenir. Eğer kullanıcı yanlış hesap numarası ve(ya) PIN girdi ise, ekranda bir mesaj görüntülenir ve ATM kimlik denetimini yeniden başlatmak için **ADIM 1'e** geri döner.



ATM Sistemi için Yazılım Geliştirme...

❑ Gereksinim Analizi Aşaması...

- ❑ ATM, kullanıcının kimlik denetimini tamamladıktan sonra, ana menü, bakiye sorgulama (seçenek 1), para çekme (seçenek 2) ve para yatırma (seçenek 3) olmak üzere üç tip işlemin her biri için numaralandırılmış bir seçenek sunar.
- ❑ Ayrıca, kullanıcının sistemden çıkabilmesi için de bir seçenek içermelidir (seçenek 4)
- ❑ Kullanıcı, daha sonra gerçekleştireceği işlemi (1, 2 veya 3'ü tuşlayarak) veya sistemden çıkmayı (4'ü tuşlayarak) seçer.
- ❑ Eğer kullanıcı bakiye görüntülemek için 1'i tuşlarsa, ekran kullanıcının hesabını görüntüler. Bunu gerçekleştirmek için, ATM'nin bankanın veri tabanından hesap bilgilerini alması gerekir.

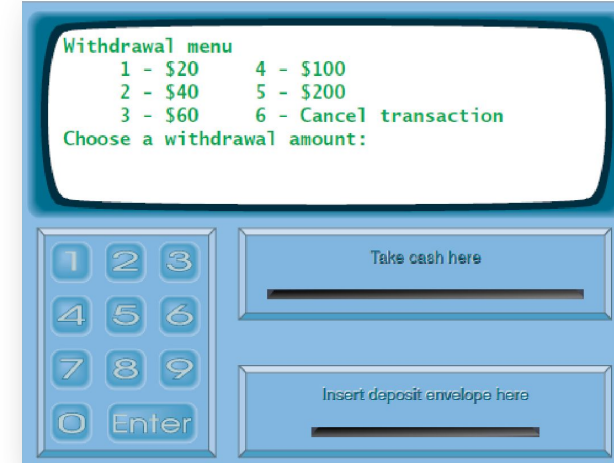


ATM Sistemi için Yazılım Geliştirme...

□ Gereksinim Analizi Aşaması...

□ Aşağıdaki adımlar kullanıcının para çekmek için 2'yi tuşlamasından sonra gerçekleşecek olanları tanımlar:

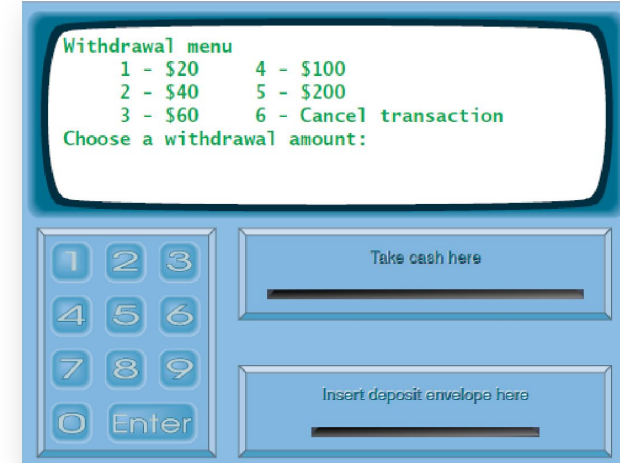
1. Ekranda, standart para çekme miktarlarını seçmek için belirlenen tuşları gösteren bir menü görüntülenir. Menü ayrıca kullanıcının işlemi iptal edebilmesine izin veren bir seçenek (seçenek 6) de içerir.
2. Kullanıcı, tuş takımını kullanarak bir menü seçimi yapar.



ATM Sistemi için Yazılım Geliştirme...

□ Gereksinim Analizi Aşaması...

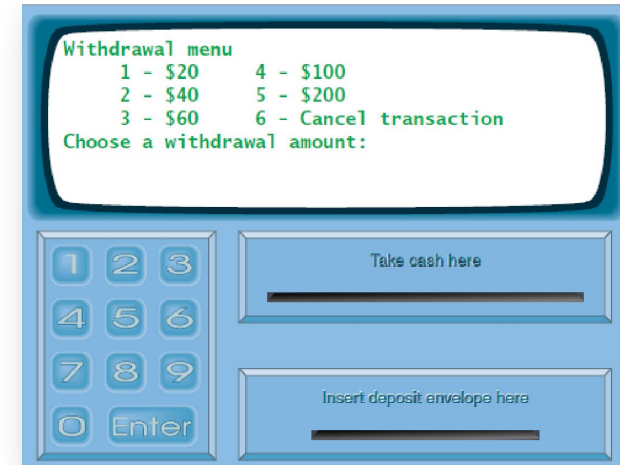
3. Eğer, çekilmek istenen para miktarı, kullanıcının hesap bakiyesinden büyükse, ekranda bu durumu belirten ve kullanıcıya daha düşük bir miktar girmesini söyleyen bir mesaj görüntülenir. ATM, daha sonra **Adım 1**'e döner. Eğer çekilmek istenen para miktarı, kullanıcının hesap bakiyesinden küçük veya eşitse (yani, kabul edilebilir bir miktarsa), ATM, **Adım 4**'e gider. Eğer kullanıcı, işlemi iptal etmeyi seçerse (seçenek 6), ATM, ana ekranı görüntüler ve kullanıcı girişini bekler.
4. Eğer, bankamatik yeterli para içeriyorsa, ATM, **Adım 5**'e gider. Aksi halde, ekranda, bankamatikte yetersiz para olduğunu belirten ve kullanıcıya daha düşük bir para çekme miktarı girmesini söyleyen bir mesaj görüntülenir. ATM, **Adım 1**'e geri döner.



ATM Sistemi için Yazılım Geliştirme...

□ Gereksinim Analizi Aşaması...

5. ATM, çekilen para miktarını bankanın veritabanındaki kullanıcı hesabından düşer (yani, kullanıcının hesap bakiyesinden çekilen para miktarını çıkartır).
6. Bankamatik, hesaplanan para miktarını kullanıcıya verir.
7. Ekranda, kullanıcıya para almasını hatırlatan bir mesaj görüntülenir.



ATM Sistemi için Yazılım Geliştirme...

□ Gereksinim Analizi Aşaması...

□ Aşağıdaki adımlar kullanıcı para yatırmak amacıyla 3'ü tuşlayınca gerçekleşen adımları tanımlar:

1. Ekran, kullanıcının yatıracağı para miktarını sorar veya iptal etmek için 0'ı tuşlamasını ister.
2. Kullanıcı, tuş takımını kullanarak yatırılacak para miktarını girer veya 0'ı tuşlar.
3. Eğer kullanıcı yatırılacak parayı girerse, ATM, Adım 4'e gider. Eğer kullanıcı işlemi iptal etmeyi seçerse (0 tuşlayarak), ATM ana ekranı görüntüler ve kullanıcı girişi için bekler.
4. Ekran, yatırılacak parayı yerleştirmesini söyleyen bir mesaj görüntüler.
5. Eğer, para yatırma bölmesi iki dakika içerisinde parayı alırsa, ATM , para miktarını bankanın veritabanındaki kullanıcının hesabına ekler. Eğer para yatırma bölmesi bu zaman diliminde yatırılacak parayı alamazsa, ekran, sistemin eylemsizlikten dolayı işlemi iptal ettiğini belirten bir mesaj görüntüler. ATM, daha sonra ana menüyü görüntüler ve kullanıcı girişi için bekler.

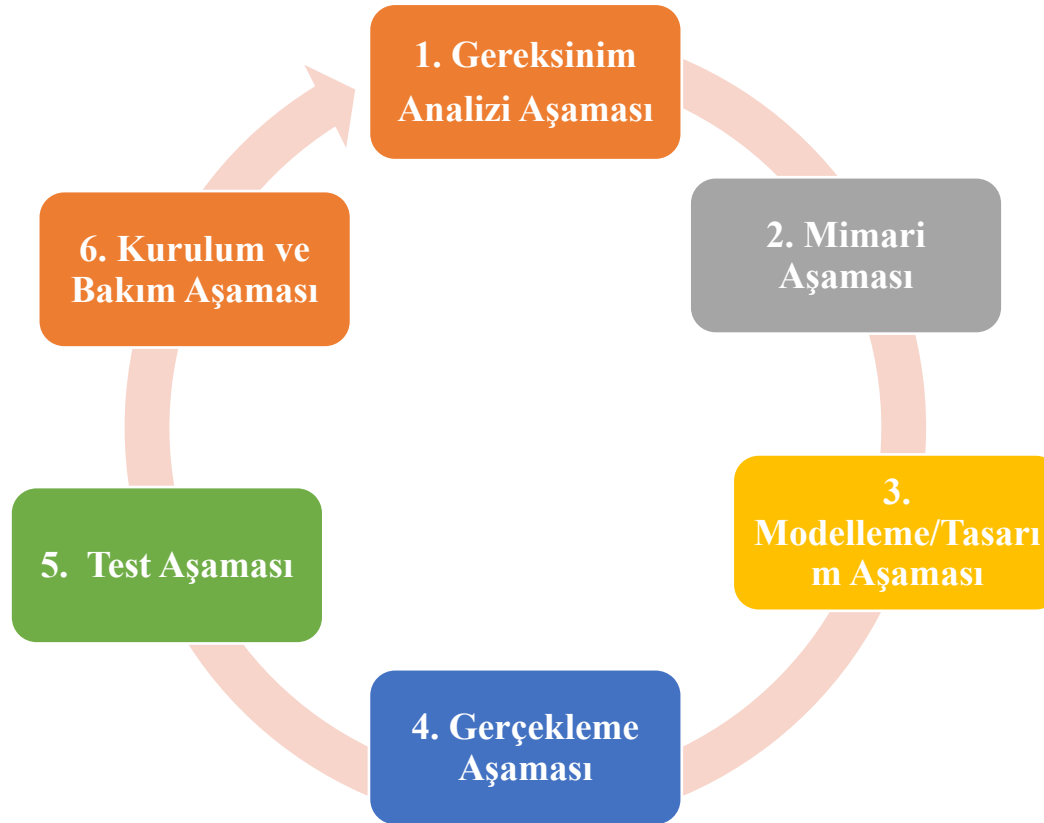
ATM Sistemi için Yazılım Geliştirme...

□ Gereksinim Analizi Aşaması...

- Sistem, başarılı şekilde bir işlemi gerçekleştirdikten sonra, ana menüye dönmelidir ki kullanıcı başka işlemler de yapabilsin. Eğer kullanıcı sistemden çıkarsa, ekran bir teşekkür mesajı görüntüler ve daha sonra sonraki yeni kullanıcı için bir hoş geldin mesajı görüntüler.

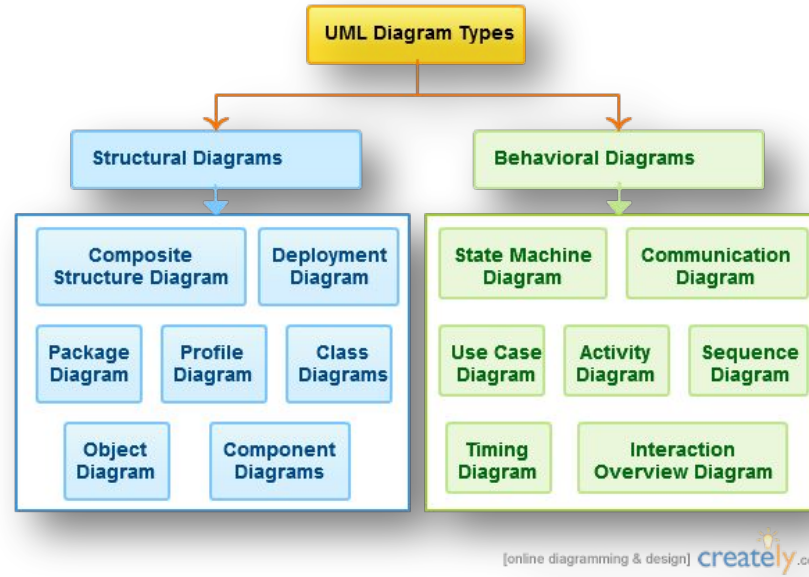
Yazılım Geliştirme Yaşam Döngüsü

□ Yazılım Geliştirme Yaşam Döngüsü



Modelleme/Tasarım Aşaması...

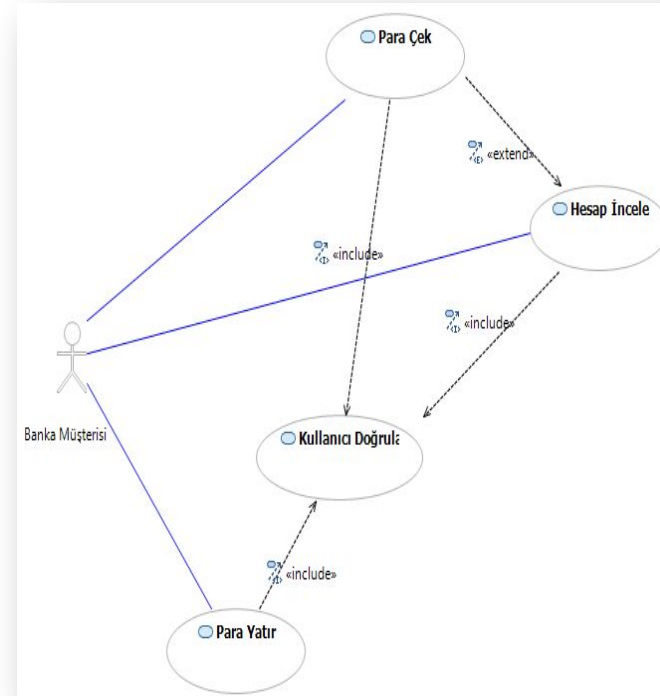
- Yazılımlar iki temel amaç için modellenirler:
 - Modelden kodlamaya geçmek daha kolaydır.
 - Farklı aktörlerin (yazılım geliştiriciler, kullanıcılar, proje yöneticileri, şirket yöneticileri v.s.) haberleşebilmeleri kolaylaşır.
- Günümüzdeki yazılımları (özellikle de nesne yönelimli olarak geliştirilenleri) modellemek için **UML** (Unified Modelling Language – Birleşik Modelleme Dili) kullanılır.



Modelleme/Tasarım Aşaması...

■ Kullanım Senaryoları (Use Case Diagram)

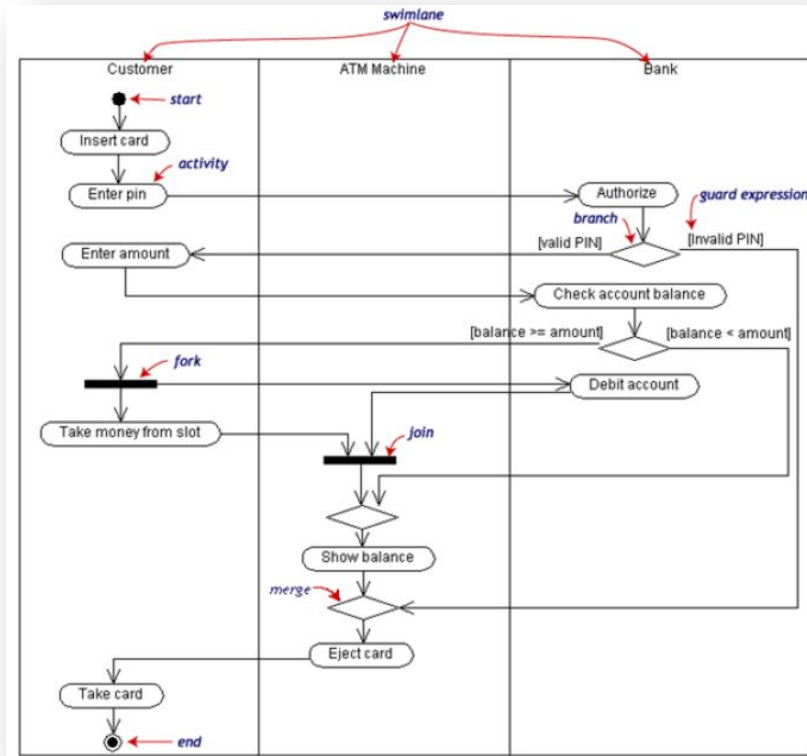
- Bir kullanıcı ve bir sistem arasındaki etkileşimi anlatan senaryo topluluğu
- Davranışsal şemalardandır.
- Aktör belirli bir görev için olayları başlatan nesnedir. (Banka müşterisi, operatör, banka v.s.)
- Kullanım senaryolarında sistem aktör açısından ne yapar ile ilgilenir. Nasıl yapar ile ilgilenmez.
- Senaryo; aktör sistem ile etkileşime girdiğinde neler olduğunu anlatır.
- **Use case**; tek bir görev ya da hedef için senaryonun özetlenmiş halidir. Use case'ler birleşerek senaryoyu oluşturur. (Para çekme bir use case dir)
- Use-case diyagramları, sistemin ne yaptığını dışarıdan bakan bir gözlemcinin bakış açısıyla anlatan diyagramlardır.



Modelleme/Tasarım Aşaması...

□ Etkinlik Şeması (Activity Diagram)

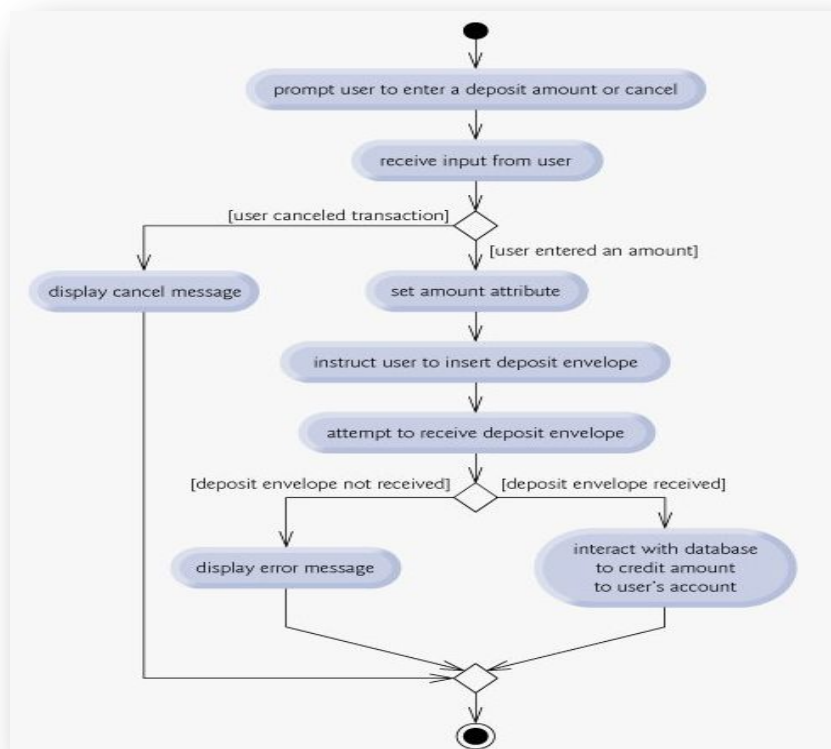
- Sistemin dinamik görünümünün modellendiği diyagramlar
- Para çekme senaryosuyla ilgili şema



Modelleme/Tasarım Aşaması...

□ Etkinlik Şeması (Activity Diagram)

- Sistemin dinamik görünümünün modellendiği diyagramlar
- Para yatırma senaryosuyla ilgili şema



Modelleme/Tasarım Aşaması...

- Sözde kod ve akış şemaları da yazılımların algoritmalarını ifade etmek için kullanılan kısıtlı modelleme yaklaşımlarıdır.
- Akış şemaları, UML'deki etkinlik şemaları gibidir. Etkinlik şeması, yapısal programlamadaki modelleme standartlarından olan veri akış şemaları (dfd – data flow diagram) ve akış şemalarının birleşiminden oluşur.

Algoritma, Sözde Kod ve Akış Diyagramı

Konu & İçerik

- Algoritma Nedir? İyi Bir Algoritmada Neler Olmalıdır?
- Büyük Resim – Algoritma Farklı Şekillerde İfadeleri
 - Algoritma, Sözde Kod, Akış Diyagramı
- Algoritma Tasarımında Temel Kavramlar/Yapılar/İşlemler
 - Değişken, Sabit, Sayaç Sorgu, Döngü
 - Matematiksel, Mantıksal ve Karşılaştırma İşlemleri
 - Algoritma Örnekleri
- Akış Diyagramlarında Kullanılan Geometrik Semboller
 - Akış Diyagramı Örnekleri
- Kaynaklar

Algoritma Nedir?

- Gerçek hayattaki "plan" kelimesinin karşılığı olarak düşünülebilir.
- Bir problemin çözümüne yönelik işlem basamaklarının sıralı biçimde ve sonlu olarak ifade edilmesi
- Bilgisayar dilinde “ bir sorunun çözümü için öngörülen işlemlerin mantıksal ve sembolik anlatımı”
- 9. yüzyılda yaşamış Türk-İslam matematikçi ve astronomu Harzemli Mehmet, toplama, çıkarma, ikiye bölme, bir sayının iki katını bulma, denklem çözümü, vb. gibi cebirsel işlemleri açıklayan bir çalışma yaptı. Batılılar, onun bu çalışmalarına Latince "algorismus" yani bugünkü adı ile **algoritma** dediler ve algoritma kavramı ilk defa burada kullanılmış oldu.
- Bir algoritma yazmak için mutlaka bir dile bağımlı kalma zorunluluğu yoktur. Önemli olan yazılan algoritmanın herhangi bir programlama diline uyarlanabilir olmasıdır.

- Algoritmanın etkin bir şekilde oluşturulması Program yazma adımından çok daha önemlidir.
- Tasarlanan algoritma iyi değilse kullanılan programlama dilinin önemi yoktur. (C, Java, Python, C++)
- Bir problemin çözümü için birbirinden farklı birden fazla sayıda algoritma hazırlanabilir.
- Herhangi bir problemin çözümü için birbirinden farklı yüzlerce bilgisayar programı yazılabilir.

Algoritma...

❑ İyi Bir Algoritmada Neler Olmalıdır?

❑ Etkinlik

❑ Sonluluk

❑ Kesinlik

❑ Giriş/Çıkış

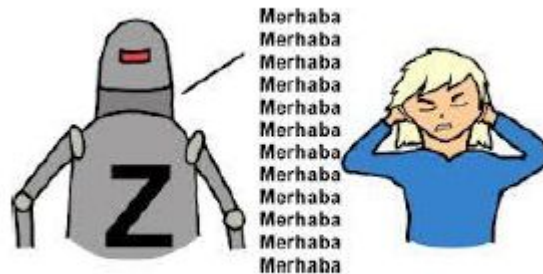
❑ Başarım/Performans

□ Etkinlik

- Yazılan algoritmalar gereksiz tekrarlardan uzak oluşturulmalıdır.
- Algoritmalar genel amaçlı yazılıp diğer algoritmalar içerisinde de kullanılabilir olmalıdır.
- Örnek: Verilen n adet sayının ortalamasını bulmakta kullandığımız algoritma varsa bu algoritma, bir sınıfta öğrencilerin yaş ortalamasını bulan bir algoritma için de kullanılabilir.

❑ Sonluluk

- ❑ Her algoritma bir başlangıçtan oluşmalı, belirli işlem adımı içermeli ve bir bitiş noktasına sahip olmalıdır.
- ❑ Olasılıklar değerlendirilerek algoritma sonlu adımda bitmelidir.
- ❑ Kısır bir döngüye girmemelidir.



□ Kesinlik

- İşlem sonucu kesin olmalı, her yeni çalıştırmada aynı sonucu üretmelidir.
- Her adımı anlaşılır, basit ve kesin bir biçimde ifade edilmiş olmalıdır.
- Yorum gerektirmemeli ve belirsiz ifadelere sahip olmamalıdır.

❑ Giriş/Çıkış

- ❑ Algoritma giriş (üzerinde işlem yapılacak değerler) ve çıkış (yapılan işlemler neticesinde üretilen sonuç değerler) değerlerine sahip olmalıdır.
- ❑ Algoritmaların temel amacı giriş bilgisini işleyerek çıkış bilgisi oluşturmaktır.
- ❑ Bazı durumlarda bir algoritmanın çıkış bilgisi başka bir algoritmanın giriş bilgisi olarak kullanılabilir.



❑ Başarım/Performans

- ❑ Amaç, donanım gereksinimi (bellek kullanımı, vs.), çalışma süresi, vs. gibi performans kriterlerini dikkate alarak yüksek başarılı programlar yazmak olmalıdır.
- ❑ Gereksiz tekrarlar ortadan kaldırılmalıdır. Bir algoritmanın performans değerlendirmesinde aşağıdaki temel kriterler göz önünde bulundurulur.
 - ❑ Birim İşlem Zamanı
 - ❑ Veri Arama ve Getirme Zamanı
 - ❑ Kıyaslama Zamanı
 - ❑ Aktarma Zamanı

□ Çay demleme algoritması?

□ Pilav yapma algoritması?

□ Zeka Küpü

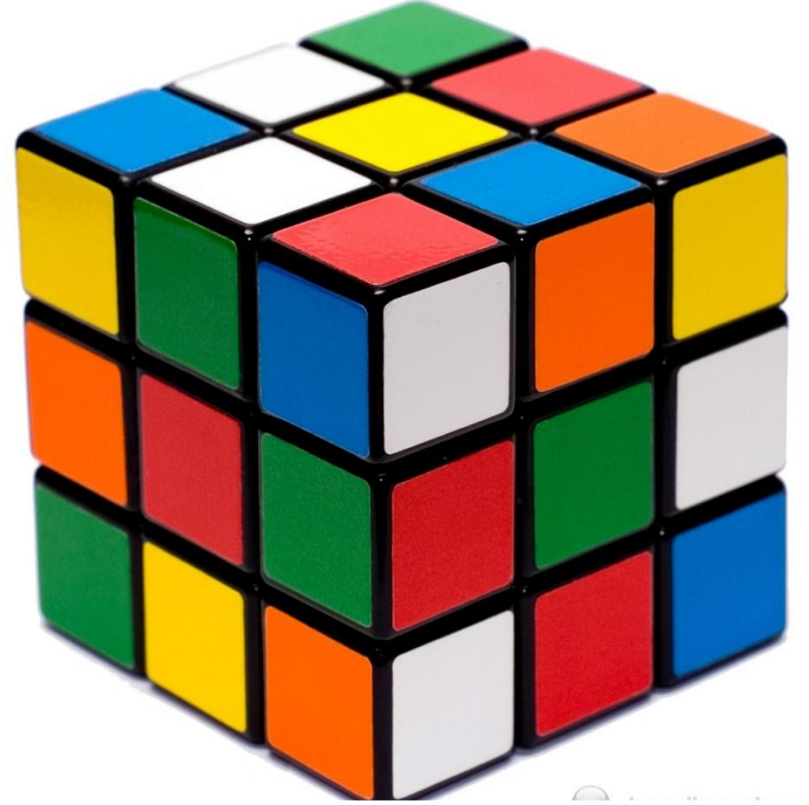
Çay Yapma Algoritması

- ❑ Mutfakta değilsen mutfığa git.
- ❑ Çayı kontrol et, çay yoksa markete git
- ❑ Çaydanlığa bak, dolu değilse su doldur.
- ❑ Ocağı yak ve çaydanlığı ateşin üstüne koy.
- ❑ Suyun kaynamasını bekle.
- ❑ Su kaynadıktan sonra çayı bırak ve üstüne suyu dök.
- ❑ Yine demliğe biraz daha su ilave ederek bekle.
- ❑ Su kaynadığında biraz dinlendirerek ateşi kapat.
- ❑ Çay bardağını al çayını doldur.
- ❑ Çayına istediğin kadar şeker at (ya da atma) ve karıştır.
- ❑ Geldiğin odaya geri dön.
- ❑ Ve çayı iç.



Zeka K p 

Zeka k p n  dođru bir Őekilde tamamlayabilir misiniz?



Algoritma

□ Algoritmalar Farklı Şekillerde İfade Edilebilir mi?

- Algoritmanın **metin** olarak yazılması

- Çözülecek problem, adım adım metin olarak yazılır.

- **Sözde/Kaba Kod** (Pseudo Code)

- Algoritmanın sözde kodlarla yazılmasıdır.

- Problemin çözüm adımları komut benzeri anlaşılır metinlerle ifade edilir.

- Yarı kod yarı metin olarak da adlandırılır.

- Sözde kod, programlar gibi derlenmez ve işlenmez.

- **Akış Diyagramı**

- Problemin çözüm adımları **geometrik şekiller** ile ifade edilir.

Algoritma...

□ Sözde Kod

- Algoritmaların ifade edilmesinde akış diyagramlarının yanı sıra, konuşma dili ile programlama dili arasında, **sözde kod** (pseudo-code) adı verilen bir araç kullanılır.
- Programlama dilinden bağımsız olarak yazılan kodlardır, yani programlar gibi derlenmez veya işlenmezler.
- Her programcı kendi sözde kodunu geliştirebilir . Fakat kişisel sözde kodlar başkaları tarafından anlaşılabilir bir biçimde açık olmalıdır.

Algoritma...

❑ ÖRNEK

❑ Problem: Klavyeden girilen sayının karesini hesaplayarak ekrana yazdıran programın algoritmasını yazınız.

❑ Algoritma:

- ❶ Başla
- ❷ Sayıyı (**A**) gir
- ❸ Sayının karesini hesapla (**Kare** = $A * A$ işlemini yap)
- ❹ Sonucu (**Kare**) yaz
- ❺ Dur

Algoritma...

❑ ÖRNEK...

❑ Problem: Klavyeden girilen sayının karesini hesaplayarak ekrana yazdıran programın sözde kodunu yazınız.

❑ Sözde Kod:

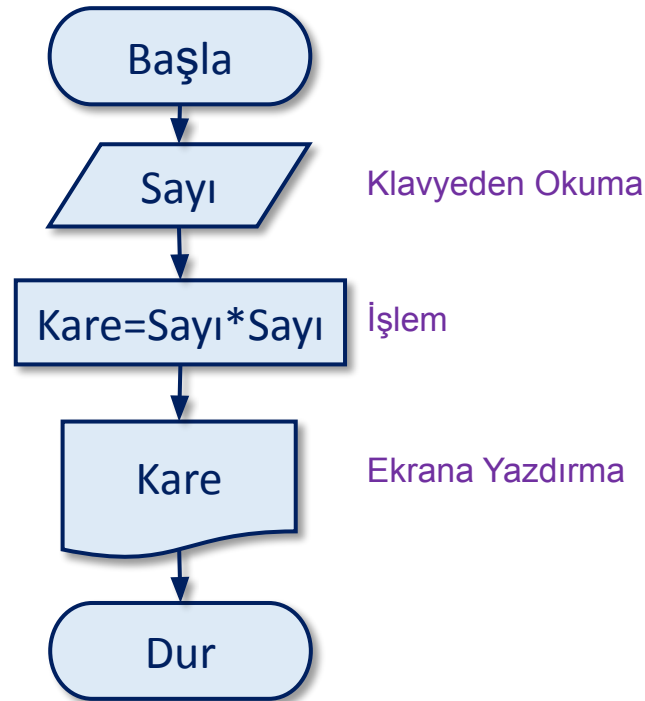
- ❶ `/* Kare alma programı */`
- ❷ `cin >> A`
- ❸ `kare=A*A`
- ❹ `cout << kare`
- ❺ `END`

Algoritma...

❑ ÖRNEK...

❑ Problem: Klavyeden girilen sayının karesini hesaplayarak ekrana yazdıran programın akış diyagramını çiziniz.

❑ Akış Diyagramı:



Algoritma Tasarımında Kullanılan Terimler

□ Operatör

□ İşlemleri belirten yani veriler üzerinde işlem yapma özelliği olan simgelerdir.

□ Örnek:

□ + = >

Algoritma Tasarımında Kullanılan Terimler...

□ Tanımlayıcı

- Programcı tarafından oluşturulan/verilen ve programdaki değişkenleri, sabitleri (değişmez), sınıf, nesne, özel bilgi tiplerini, alt programları vb. adlandırmak için kullanılan kelimelerdir.
- Tanımlayıcı, yerini tutacağı ifadeye çağrışım yapmalıdır.
- **Tanımlayıcı kelimeler oluşturulurken uyulması gereken kurallar:**
 - İngiliz alfabesindeki A-Z veya a-z arası 26 harf kullanılabilir
 - 0-9 arası rakamlar kullanılabilir
 - Simgelerden sadece alt çizgi (_) kullanılabilir
 - Tanımlayıcı isimleri, harf veya alt çizgi ile başlayabilir
 - Rakam ile başlayamaz veya sadece rakamlardan oluşamaz
 - Kullanılan programlama dilinin komutu ya da saklı kelimelerinden olamaz

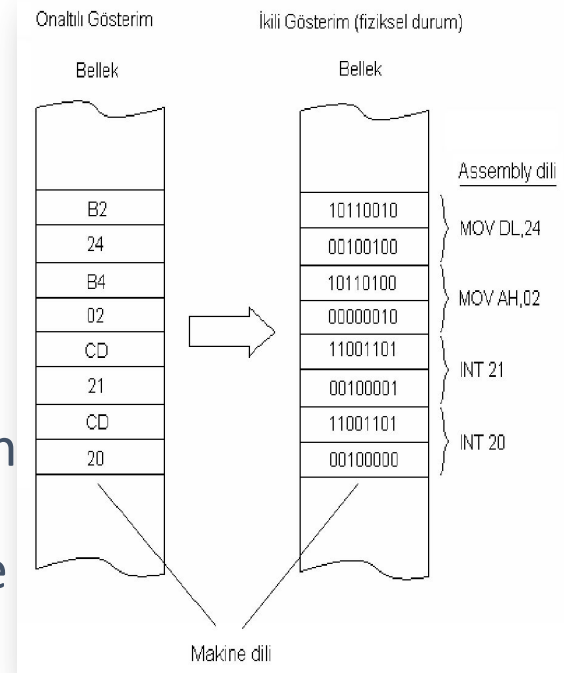
Algoritma Tasarımında Kullanılan Terimler...

Değişken

- Verilerin saklandığı bellek alanlarına verilen simgesel isimlerdir.
- Programın her çalıştırılmasında, farklı değerler alabilen/aktarılabilen bilgi/bellek alanlarıdır.
- Bir programda birbirinden farklı kaç tane veri tutulacak ise o kadar değişken tanımlanmalıdır.
- Değişkenleri adlandırma, tamamen programcının isteğine bağlıdır.
- Değişken adlandırmada tanımlayıcı isimlendirme kuralları geçerlidir.
- Değişken adının, yerini aldığı ifadeyi çağrışım yapması programın anlaşılabilirliği açısından önemlidir.

Örnek:

- Bir kişinin adını tutmak için **ad** , telefonunu tutmak için **telNo**



Algoritma Tasarımında Kullanılan Terimler

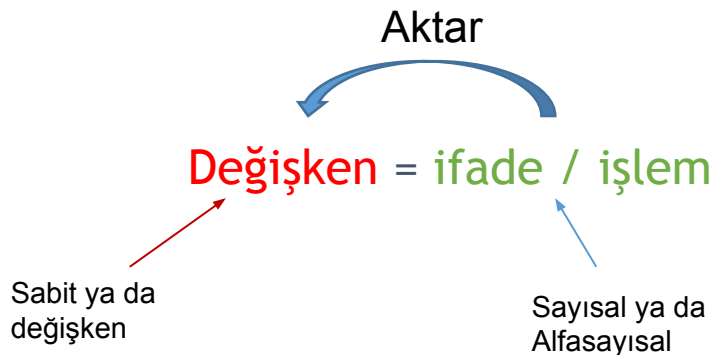
❑ Sabit

- ❑ Programlardaki değeri değişmeyen ifadelere sabit denir.
- ❑ Sabit adlandırmalarında da tanımlayıcı kuralları geçerlidir.
- ❑ Sabitlere değer aktarma
 - ❑ Sayısal veriler doğrudan aktarılır.
 - ❑ Örnek: SabitKomutu $\pi = 3.14$
 - ❑ Alfasayısal (karakter) veriler ise tek/çift tırnak içerisinde aktarılır.
 - ❑ Örnek: SabitKomutu ilkharf = 'A'
 - ❑ Örnek: SabitKomutu okulAdi = "Sakarya"

Algoritma Tasarımında Kullanılan Terimler

□ Aktarma

- Bir bilgi alanına, veri yazma
- Bir ifadenin sonucunu başka bir değişkende gösterme vb. görevlerde **aktarma** “=” operatörü kullanılır.
- Kullanım şekli:



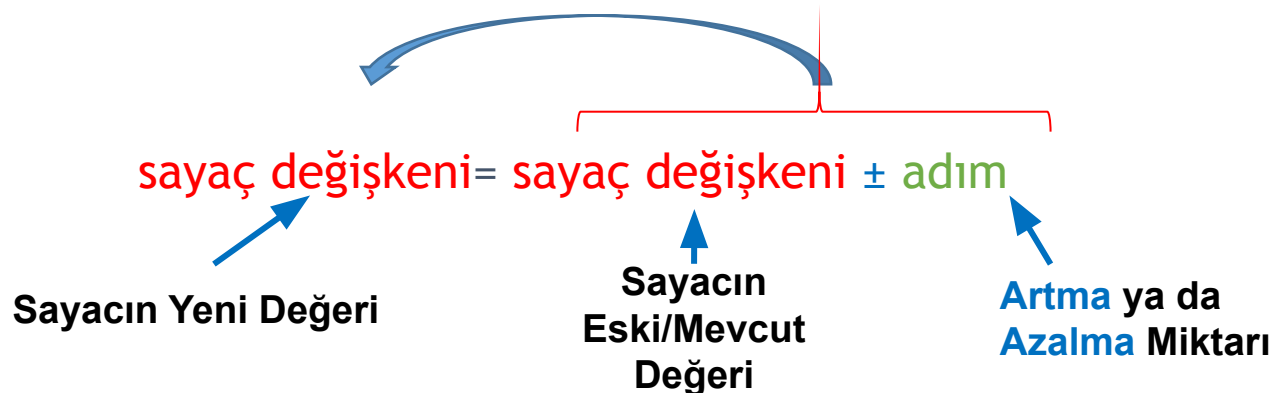
Örnek:

- Sayısal ifade olarak
 - $A = 2, B = 3$
 - $C = A + B$
- Alfasayısal ifade olarak
 - $A = \text{"Sak"}, B = \text{"arya"}$
 - $C = A + B$

Algoritma Tasarımında Kullanılan Terimler...

□ **Sayaç / Sayıcı**

- Belirli sayıda yapılması istenen işlemleri takip etmek için
- İşlenen ya da üretilen değerlerin sayılması gerektiği durumlarda kullanılır.
- Örnek: Rasgele oluşturulmuş bir dizideki tek sayıların tespitinde tek sayıların adedini belirlemek için sayaç kullanılır.
- Kullanım şekli:



Örnek:

□ **Say = Say + 1**

Algoritma Tasarımında Kullanılan İşlemler

□ Matematiksel (Aritmetik) İşlemler

İşlem	Matematik	Bilgisayar
Toplama	$a + b$	$a + b$
Çıkarma	$a - b$	$a - b$
Çarpma	$a \cdot b$	$a * b$
Bölme	$a \div b$	a / b
Üs Alma	a^b	$a ^ b$

Algoritma Tasarımında Kullanılan İşlemler

□ Matematiksel (Aritmetik) İşlemler...

Matematiksel Yazılım

$$a+b-c+2abc-7$$

$$a+b^2-c^3$$

Bilgisayara Kodlanması

$$a+b-c+2*a*b*c-7$$

$$a+b^2-c^3$$

$$a-b/c+2*a*c-2/(a+b)$$

$$(a+b)^{(1/2)}-2/(b^2-4*a*c)$$

$$(a^2+b^2)/(2*a*b)$$

Algoritma Tasarımında Kullanılan İşlemler...

❑ Matematiksel (Aritmetik) İşlemler...

Sıra	İşlem	Bilgisayar
1	Sayıların Negatifliği	-...
2	Parantezler	(.....)
3	Matematiksel Fonksiyonlar	<i>cos, sin, log, ...</i>
4	Üs alma	$a ^ b$, <i>pow, ...</i>
5	Çarpma ve Bölme	$a * b$ ve a/b
6	Toplama ve Çıkarma	$a + b$ ve $a - b$

➤ Örnek:

❑ Matematiksel ifade :

$$x = a.b / c + d.e^f - g$$

❑ Bilgisayar ifadesi:

$$x = a * b/c + d * e^f - g$$

7 2 3 5 4 1 6

Algoritma Tasarımında Kullanılan İşlemler...

❑ Karşılaştırma İşlemleri

- ❑ İki büyüklükten hangisinin büyük veya küçük olduğu,
- ❑ İki değişkenin birbirine eşit olup olmadığı gibi konularda karar verebilir.
- ❑ Karşılaştırma İşlemlerinin Bilgisayar Dilindeki Karşılıkları

Sembol	Anlamı
==	Eşittir
<> Veya !=	Eşit Değildir
>	Büyüktür
<	Küçüktür
>= veya =>	Büyük eşittir
<= veya =<	Küçük eşittir

❑ Örnek:

Eğer **A > B** ise Yaz “ A sayısı B’den büyüktür ”

Algoritma Tasarımında Kullanılan İşlemler...

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

Algoritma Tasarımında Kullanılan İşlemler...

□ Mantıksal İşlemler

□ Temel Mantıksal İşlem Karşılıkları

İşlem	Komut	Anlamı
VE	AND	Koşulların hepsi doğru ise sonuç doğrudur
VEYA	OR	Koşullardan en az biri doğru ise sonuç doğrudur
DEĞİL	NOT	Sonuç koşulun tersidir . 1 ise 0 dır

□ Mantıksal İşlemlerde Öncelik Sıraları

Sıra	İşlem	Komut
1	Parantez İçindeki İşlemler	(.....)
2	DEĞİL	NOT
3	VE	AND
4	VEYA	OR

Algoritma Tasarımında Kullanılan İşlemler...

□ Mantıksal İşlemler...

- Örnek: Bir işyerinde çalışan işçiler arasından yalnızca yaşı 23 üzerinde olup, maaş olarak asgari ücret alanların isimleri istenebilir.
- Burada iki koşul vardır ve bu iki koşulun da doğru olması gerekir. Yani;

EĞER Yaş>23 VE maaş=asgari ücret ise YAZ

1.KOŞUL 2.KOŞUL

- *YAZ* komutu 1. ve 2.koşulun her ikisi de sağlanıyorsa çalışır

Algoritma Tasarımında Kullanılan İşlemler...

□ Mantıksal İşlemler...

□ Örnek: Bir sınıfta Programlama dersinden 65 in üzerinde not alıp, Türk Dili veya Yabancı Dil derslerinin herhangi birinden 65 in üzerinde not alanların isimleri istenmektedir.

□ Burada 3 koşul vardır.

- Programlama dersinden 65 in üzerinde not almış olmak temel koşuldur.
- Diğer iki dersin notlarının herhangi birinin 65 in üzerinde olması gerekir.

EĞER Prog_not>65 VE (TDili_not>65 VEYA YDil_not>65) ismi YAZ

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Sorgu / Seçme

- Algoritmada, işlemler genellikle sıralı adımlardan oluşur.
- Bazı koşul/şart durumlarında işlem sıralarının değiştirilmesi ve diğer bir işlem sırasının seçilmesi yada programın yeni işlem sırasından devam etmesi gerekebilir.
- İşlem sıralarının değiştirilmesi ya da bazı koşulların sorgulanması için '**EĞER**' sorgu deyimi kullanılır.
- Örnek:

EĞER $\underbrace{\text{ORT} > 50}_{\text{KOŞUL}}$ **GİT** $\underbrace{\text{ADIM 7}}_{\text{Gidilecek İşlem Sırası}}$

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Sorgu / Seçme

□ Örnek: Dışarıdan girilen iki sayıyı kıyaslayan programın algoritmasını çıkarınız?

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Sorgu / Seçme

□ Örnek: Dışarıdan girilen iki sayıyı kıyaslayan programın algoritmasını çıkarınız?

- ① Başla
- ② OKU X, Y
- ③ Eğer $X > Y$ ise YAZ “X BÜYÜK” Git 6
- ④ Eğer $X < Y$ ise YAZ “X KÜÇÜK” Git 6
- ⑤ YAZ “EŞİT”
- ⑥ Dur

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Sorgu / Seçme...

□ Örnek: Dışarıdan 10'dan büyük 20'den küçük sayı girilene kadar programı devam ettiren algoritmayı yazınız.

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Sorgu / Seçme...

□ Örnek: Dışarıdan 10'dan büyük 20'den küçük sayı girilene kadar programı devam ettiren algoritmayı yazınız.

- ① Başla
- ② OKU X
- ③ Eğer ($X > 10$ VE $X < 20$) ise Git 6
- ④ YAZ “ SAYI 10 İLE 20 ARASINDA DEĞİLDİR”
- ⑤ GİT 2
- ⑥ YAZ “SAYI 10 İLE 20 ARASINDADIR”
- ⑦ Dur

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Döngü

- Bazı işlemleri belirli sayıda tekrar etmede,
- Belirli bir aralıktaki ardışık değerler ile işlem yapmada döngü kullanılır.
- Diğer bir deyişle, programdaki belirli işlem bloklarını, verilen sayıda gerçekleştiren işlem akış çevrimlerine **DÖNGÜ** denir.

□ Örnek:

- 1'den 5'e kadar sayıların toplamı
- Ard arda ekrana 4 defa SAKARYA yazdırma
- 1 ile 100 arasındaki çift sayıların ya da tek sayıların toplamı

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Döngü...

□ Döngü oluşturma kuralları

- ❶ Döngü değişkenine başlangıç değeri verilir
- ❷ Döngünün artma ya da azalma miktarı belirlenir
- ❸ Döngünün bitiş değeri belirlenir
- ❹ Eğer Döngü karar ifadeleriyle oluşturulduysa; döngü değişkeni, döngü içinde adım miktarı kadar arttırılmalı/azaltılmalıdır.

□ Örnek:

- ❶ Başla
- ❷ $T = 0$
- ❸ $J = 1$
- ❹ Eğer $J > 10$ ise Git 8
- ❺ $T = T + J$
- ❻ $J = J + 2$
- ❼ Git 4
- ❽ Yaz T
- ❾ Dur

Algoritma Tasarımında Kullanılan İşlemler/Yapılar

□ Döngü...

□ Döngü oluşturma kuralları

- ① Döngü değişkenine başlangıç değeri verilir
- ② Döngünün artma ya da azalma miktarı belirlenir
- ③ Döngünün bitiş değeri belirlenir
- ④ Eğer Döngü karar ifadeleriyle oluşturulduysa; döngü değişkeni, döngü içinde adım miktarı kadar arttırılmalı/azaltılmalıdır.

□ Örnek: 1-10 arası tek sayıların toplamı hesaplayan programın algoritmasını çıkarınız?

- ① Başla
- ② $T = 0$
- ③ $J = 1$
- ④ Eğer $J > 10$ ise Git 8
- ⑤ $T = T + J$
- ⑥ $J = J + 2$
- ⑦ Git 4
- ⑧ Yaz T
- ⑨ Dur

Algoritma Örnekleri - 1

- Problem: Klavyeden okunan bir reel sayının karekökünü bulup sonucu ekrana yazan bir algoritmanın tasarlanması.
- Tasarım: öncelikle problemin çözümünün matematiksel olarak ifade edilmesi gerekmektedir;
- a , karekökü bulunmak istenen sayı olsun, x değeri a 'nın tahmini karekökü ve b değeri ise a 'nın gerçek karekökü ile tahmin edilen karekökü arasındaki fark olsun. Bu durumda a aşağıdaki şekilde ifade edilebilir;
 - $a = (x+b)^2 \Rightarrow a = x^2 + 2xb + b^2$
- Küçük olması beklenen b^2 değeri ihmal edilirse, b değeri yaklaşık olarak hesaplanabilir;
 - $b \approx (a - x^2) / 2x$

Algoritma Örnekleri – 1...

- Hesaplanan b değeri kullanılarak a 'nın kareköküne daha yakın yeni bir tahmin yapılabilir;
- $x_{i+1} = x_i + b$
 - Burada x_i önceki tahmin, x_{i+1} ise kareköke yakın yeni tahmin değeridir
- Bu şekilde a 'nın karekökü girilerek yakınsayan bir iterasyon (tekrarlama) ile bulunabilir.
- a 'nın karekökünü yakınsayarak bulan bu iteratif (mutlak hata $|b|$, ε hata değerinden küçük eşit olana kadar işlem tekrar edilecek).

Algoritma Örnekleri – 1...

□ Algoritma sözde kodlar ile ifade edildiğinde aşağıdaki şekilde yazılabilir (ifade kolaylığı için x_i yerine x ve x_{i+1} yerine y kullanılmıştır)

A1: Başla
A2: Oku (a) // karekökü bulunacak sayıyı a değişkenine oku
A3: Oku (x) // ilk tahmini karekökü x değişkenine oku
A4: Oku (ϵ) // kabul edilebilir hata değerini ϵ değişkenine oku
A5: $b = (a - x^2) / 2x$ // fark (hata) değeri olan b' yi hesapla
A6: $y = x + b$ // daha yakın yeni karekök değerini (y) hesapla
A7: Eğer $|b| \leq \epsilon$ ise A10'a git // $|b| \leq \epsilon$ ise iterasyonu durdurmak için A10'a git
A8: $x = y$ // y yeni karekök değerini x değişkenine ata
A9: A5'e git // işlemi yeni x tahmini ile tekrarlamak için A5'e git
A10: Yaz (y) // en son hesaplanan karekök değerini (y) ekrana yaz
A11: Dur // programı sonlandır

Algoritma Örnekleri – 1...

- Bu algorithmada işlemlerin bir çevrimin içinde tekrarlandığı ve istenilen hassasiyete ulaşıldığında çevrimin dışına çıkılarak işlemin tamamlandığı görülmektedir. Bilgisayar da program işletilirken bir değişkene yeni bir değer verildiğinde eski taşıdığı değer kaybolacağı not edilmelidir.
- Aşağıda bu algoritmanın nasıl çalıştığı, işlemlerin her tekrarında (çevrimin her adımında) değişkenlerin aldığı değerler bir çizelgede verilerek açıklanmıştır. a , x ve ε değerlerinin sırası ile 31.8, 5.0 ve 0.005 olarak okunduğu kabul edilsin.

Çevrim adım no	a	x	ε	b	y
	31.8	<u>5.0</u>	0.005		
1	<u>31.8</u>	5.0	0.005	0.68	5.68
2	<u>31.8</u>	5.68	0.005	-0.04	<u>5.64</u>
3	<u>31.8</u>	5.64	0.005	-0.0001	<u>5.64</u>

- Üçüncü çevrim adımında $|b|$ değeri ε değeri olan 0.005' den küçük olduğu için yeni karekök değeri hesaplanmaz en son hesaplanan karekök değeri $y=5.64$ olarak kalır ve işlem sonlandırılır.

Akış Diyagramı

- Akış diyagramı, algoritmaların geometrik şekillerle ortaya konulmasıdır.
- Akış diyagramı, problemin çözümü için yapılması gerekenlerin, başından sonuna kadar geometrik şekillerden oluşan semboller ile ifade edilmesidir.
- Her simge genel olarak yapılacak bir işi veya komutu gösterir.
- Algoritma doğal dille yazıldığı için herkes tarafından anlaşılmayabilir ancak akış diyagramlarında her bir şekil standart bir anlam taşıdığı için farklı yorumlanmaz.

Akış Diyagramında Kullanılan Semboller

■ Başla

- Programın nereden başlayacağını belirtir. Standart olarak her bağımsız algorithmada bir tane bulunur

BAŞLA

■ Dur

- Programın nerede sonlanacağını belirtir. Birden fazla olabilir. Mümkün ise sadece bir tane dur simgesi kullanılmalıdır.

DUR

■ Giriş

- Bilgisayara dışarıdan bilgi girişini temsil eder. Bu sembolün içine dışarıdan girilen bilgilerin aktarılacağı değişkenler yazılır.

a,b,c

Akış Diyagramında Kullanılan Semboller...

❑ Çıkış

- ❑ Ekran veya yazıcıya bilgi göndermeyi temsil eder



❑ İşlem

- ❑ Programın işlemesi sırasında yapılacak işlemleri ifade etmek için kullanılır



❑ Karşılaştırma (sorgu)

- ❑ Verilen koşulun sonucuna göre farklı işlem yapılacağını ifade etmek için kullanılır.



Akış Diyagramında Kullanılan Semboller...

■ Döngü

- Belirli bir işin veya bir grup işin birden çok yinelenmesi gerektiğinde kullanılır. Döngüdeki çevrim sayısı, döngü sayacı ve sayaç artırımı açıkça yazılır.



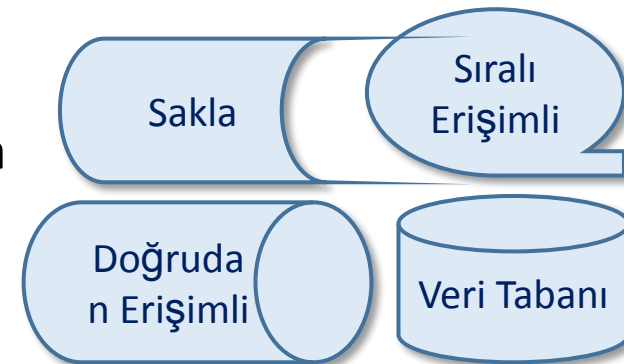
■ Fonksiyon Çağırma

- Daha önce oluşturulmuş bir algoritmanın yazılan algoritma içerisine konulmadan çağrılarak kullanılmasını ifade eder.



■ Dosyaya Saklama

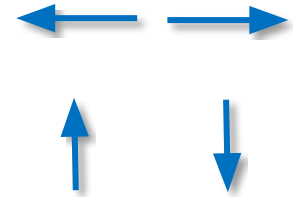
- Elde edilen bilgilerin bir dosyada saklanması veya dosyadan okunmasını simgeler.



Akış Diyagramında Kullanılan Semboller...

□ Akış Yönü

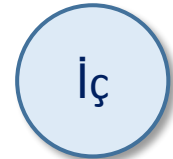
- Bir işlem bittikten sonra akışın nereye yönleneceğini belirler.



□ Bağlantı

- Akış diyagramı çizilirken sayfaya sığmama durumunda çizimin başka bir yerden devam etmesi için kullanılır.

Aynı sayfaya

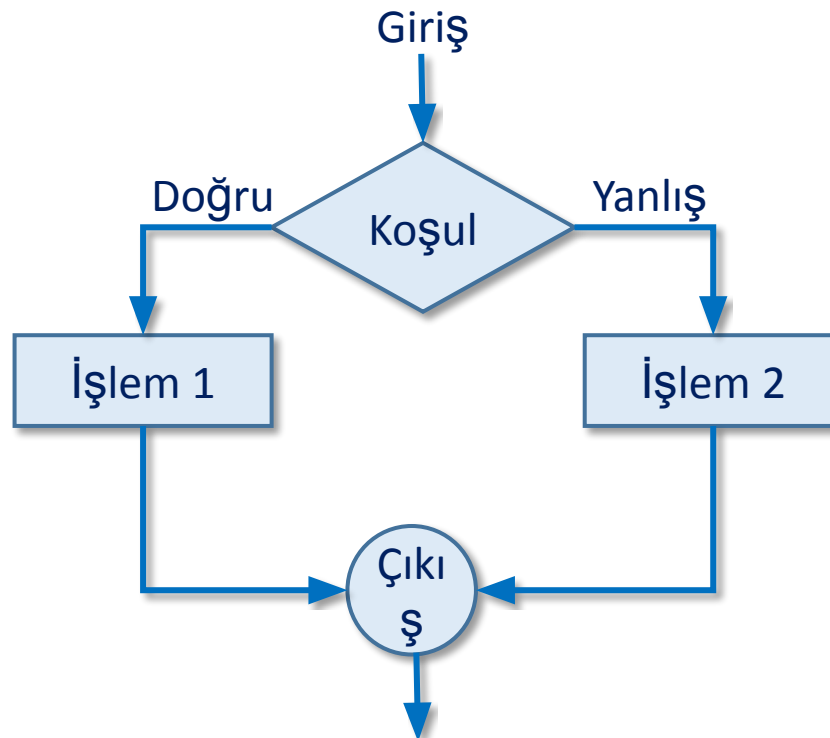


Ayrı sayfaya



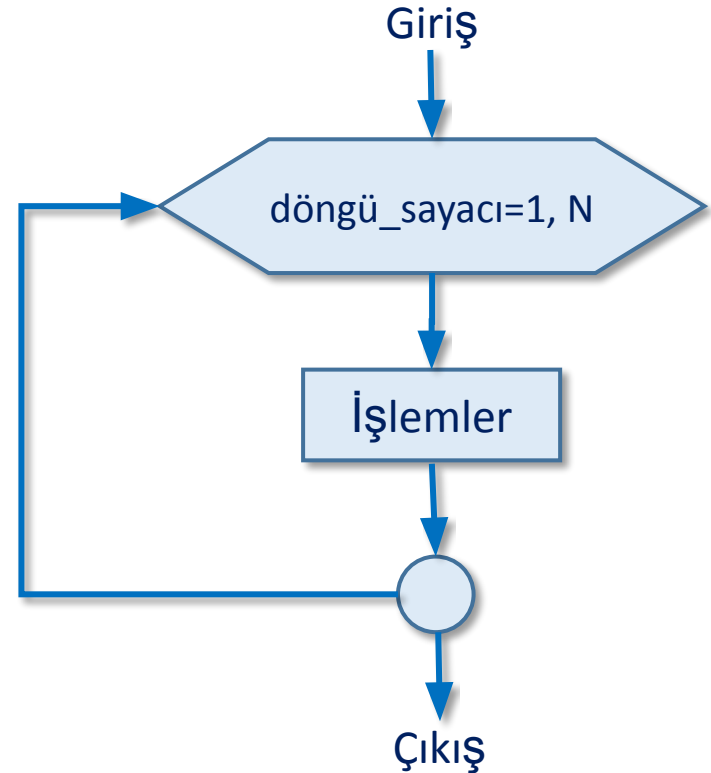
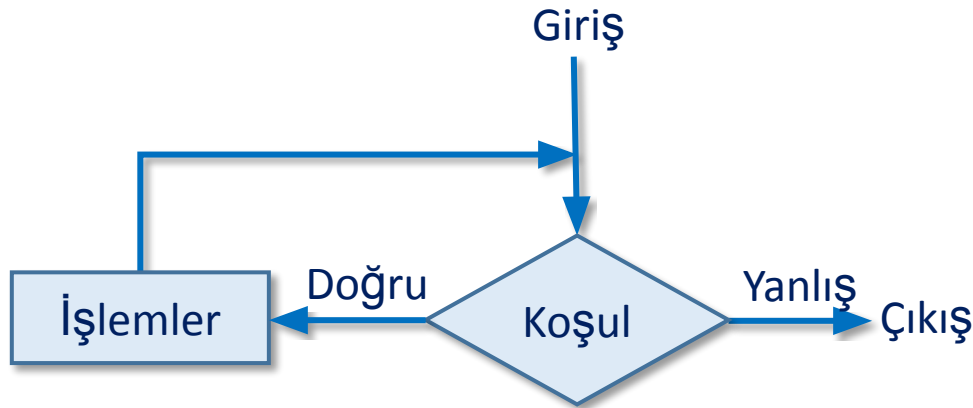
Akış Diyagramı ile Karar / Karşılaştırma Yapısı

□ Algoritma tasarımındaki **EĞER** yapısının gerçekleştirilmesi



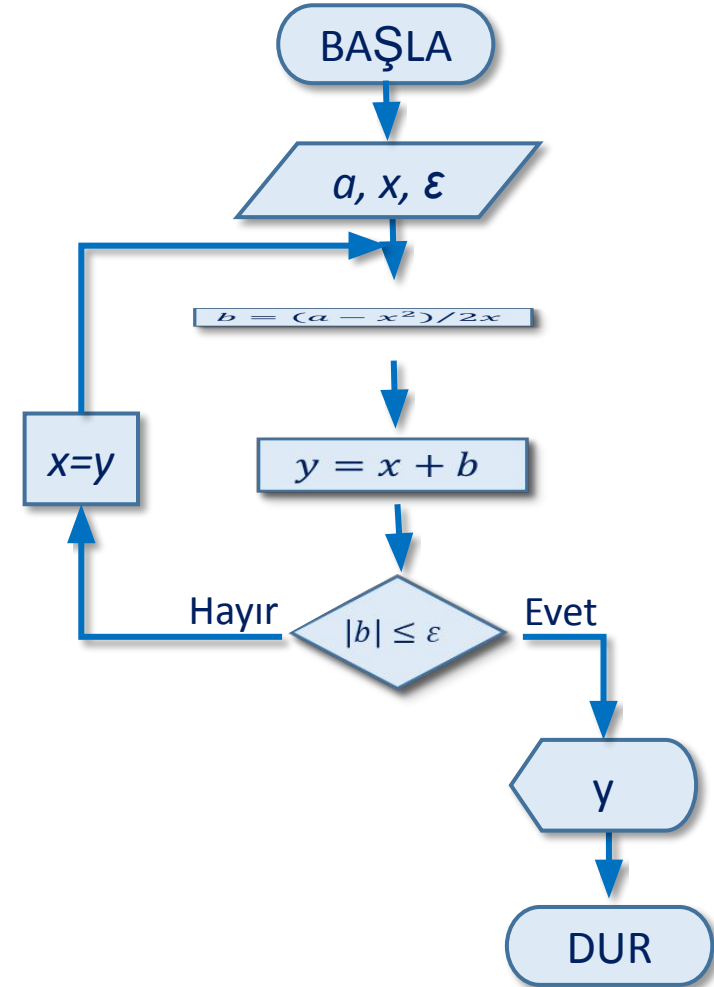
Akış Diyagramı ile Tekrar (Döngü) Yapısı

□ Algoritma tasarımındaki **Döngü** yapısının gerçekleştirilmesi



Akış Diyagramı Örnekleri - 1

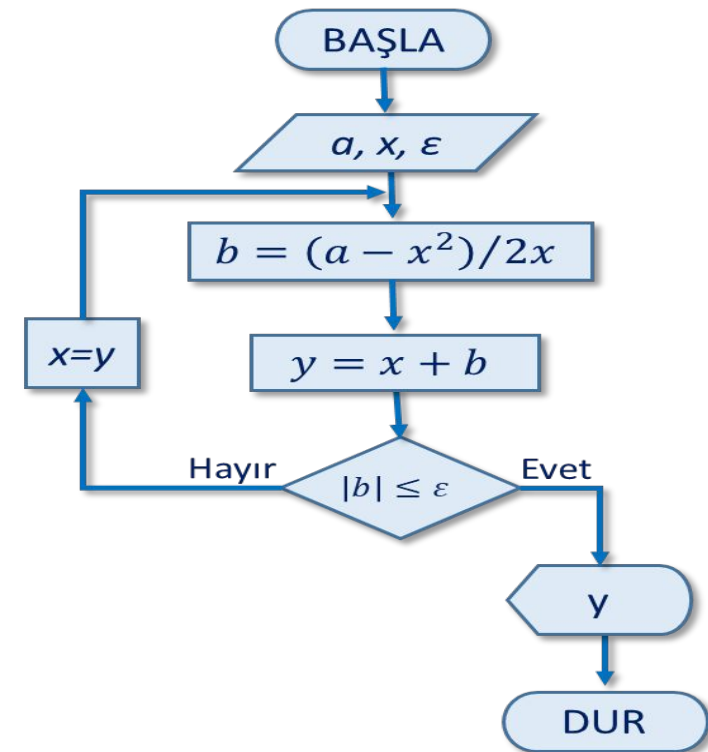
- **Problem:** Klavyeden okunan bir reel sayının karekökünü bulup sonucu ekrana yazan programın akış diyagramını çiziniz.
- Görüldüğü gibi akış diyagramlarında işlem sırası oklarla gösterildiği için deyimlerin (işlemlerin) A1, A2, vb. etiketlenmesine gerek yoktur. Fakat istenirse bu etiketler kutuların dışında sol üst köşeye yazılabilir.
- **Not:** Bu örneğin algoritma tasarımı önceki slaytlarda verilmektedir.



Akış Diyagramı Örnekleri - 1

□ Algoritma sözde kodlar ile ifade edildiğinde aşağıdaki şekilde yazılabilir (ifade kolaylığı için x_i yerine x ve x_{i+1} yerine y kullanılmıştır)

- A1: Başla
- A2: Oku (a)
- A3: Oku (x)
- A4: Oku (ϵ)
- A5: $b = (a - x^2) / 2x$
- A6: $y = x + b$
- A7: Eğer $|b| \leq \epsilon$ ise A10'a git
- A8: $x = y$
- A9: A5'e git
- A10: Yaz (y)
- A11: Dur



Akış Diyagramı Örnekleri - 2

□ Problem: 1'den N'ye kadar olan sayıların toplamını hesaplayan programın akış diyagramını çizelim.

Akış Diyagramı Örnekleri - 2

- Problem: 1'den N'ye kadar olan sayıların toplamını hesaplayan programın akış diyagramını çizelim.
- 1'den N'ye kadar, N adet sayı vardır. Birer artan döngü içinde sayıları toplayabiliriz. Döngü artışını kontrol edeceğimiz değişken i olsun. Toplam değerini de T değişkeni ile ifade edelim. Döngü değişkeni i , 1'den başlayacak ve birer artarak N ye ulaşacak. T başlangıçta 0 ile başlayacak ve döngü içerisinde 1'den N'ye değişen i değeri ilave edilecek.

Akış Diyagramı Örnekleri – 2...

□ Problemin Algoritması:

□ Algoritma sözde kodlar ile ifade edildiğinde aşağıdaki şekilde yazılabilir:

Adım 1: Başla

Adım 2: Oku (N)

Adım 3: $T = 0, i = 0$

Adım 4: $T = T + i$

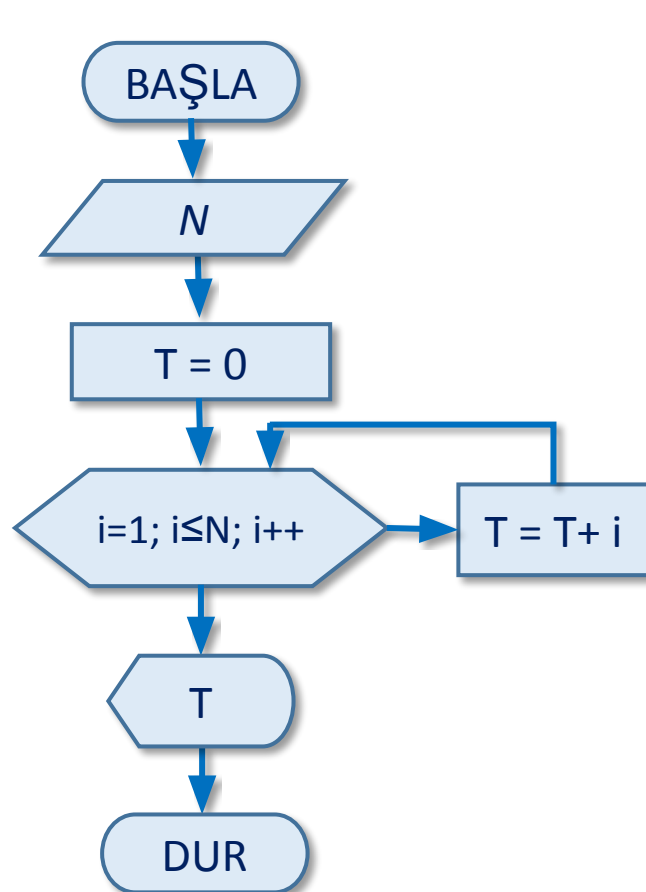
Adım 5: $i = i + 1$

Adım 6: Eğer $i \leq N$ ise Adım 4'ye git

Adım 7: Yaz (T)

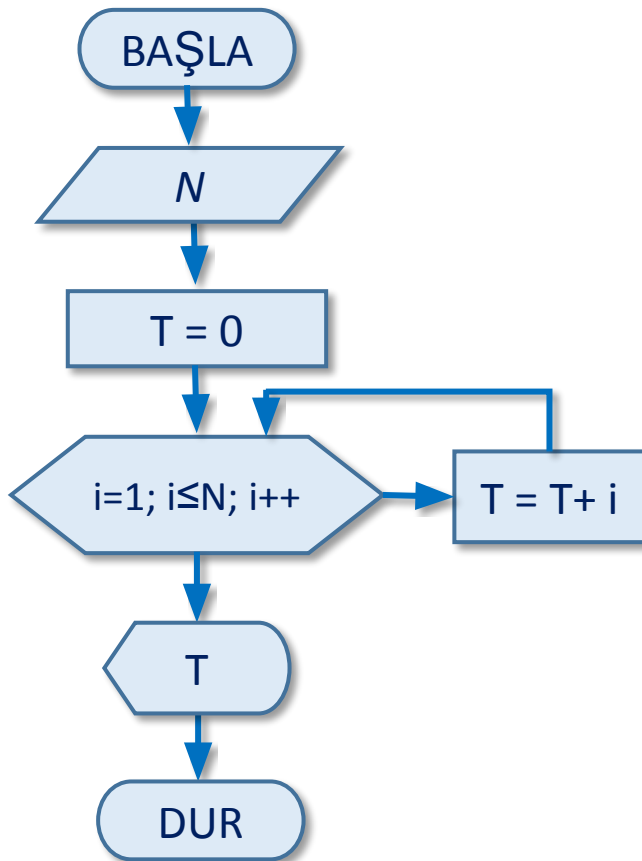
Adım 8: Dur

Akış Diyagramı Örnekleri – 2...



i N işlem T

Akış Diyagramı Örnekleri – 2...



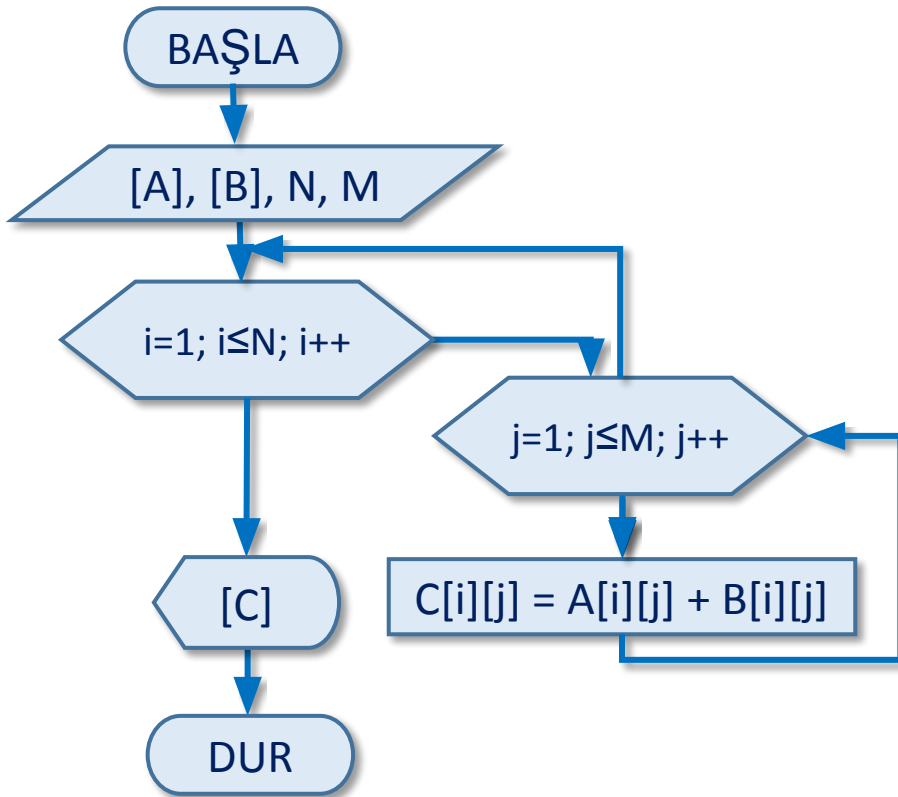
i	N	işlem	T
-	10	-	-
-	10	T=0	0
1	10	T=0+1	1
2	10	T=1+2	3
3	10	T=3+3	6
4	10	T=6+4	10
5	10	T=10+5	15
6	10	T=15+6	21
7	10	T=21+7	28
8	10	T=28+8	36
9	10	T=36+9	45
10	10	T=45+1 0	55

Akış Diyagramı Örnekleri – 3

➤ **Problem:** $N \times M$ boyutlu iki matrisin toplamını hesaplayan bir algoritmanın akış diyagramını çizelim.

- Matrislerin elemanlarını ifade eden indisleri i ve j ile gösterelim ($i=1, \dots, N$, $j=1, \dots, M$). Bu durumda A matrisinin her bir elemanı matematiksel olarak $A_{i,j}$ veya programlama açısından $A[i][j]$ şeklinde ifade edilebilir.
- Elemanları toplanacak matrisler A ve B matrisleri ve toplam sonucunda oluşacak matris ise C matrisi olsun.
- Bilindiği gibi matris toplamında birinci matrisin $[i][j]$ indeksli elemanı ile ikinci matrisin $[i][j]$ indeksli elemanı karşılıklı olarak toplanarak toplam matrisin $[i][j]$ indeksli elemanını elde edilir.

Akış Diyagramı Örnekleri – 3...



$$[A] = \begin{bmatrix} 1 & 3 \\ 1 & 2 \end{bmatrix} \quad [B] = \begin{bmatrix} 2 & 1 \\ 3 & 0 \end{bmatrix}$$

matrisleri ve $N=M=2$ değerleri için

i	j	işlem	C[i][j]
1	1	$C[1][1]=1+2$	3
1	2	$C[1][2]=3+1$	4
2	1	$C[2][1]=1+3$	4
2	2	$C[2][2]=2+0$	2

$$[C] = \begin{bmatrix} 3 & 4 \\ 4 & 2 \end{bmatrix}$$

Programlamaya Giriş

Algoritma, Sözde Kod ve Akış Diyagramı ÖRNEKLERİ

ÖRNEK

□ Ekrana "Merhaba Dünya" yazan programın algoritmasını yazınız.

ÖRNEK

□ Ekrana "Merhaba Dünya" yazan programın algoritmasını yazınız.

1. BAŞLA
2. YAZ "Merhaba Dünya"
3. DUR

ÖRNEK

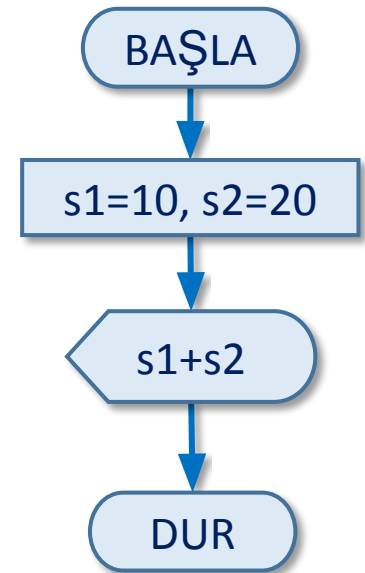
□ İki sayıyı toplayıp sonucunu ekrana yazdıran programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. $s1=10$, $s2=20$
3. YAZ $s1+s2$
4. DUR

ÖRNEK

İki sayıyı toplayıp sonucunu ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. $s1=10, s2=20$
3. YAZ $s1+s2$
4. DUR



ÖRNEK

- Klavyeden girilen iki sayıyı toplayıp sonucunu ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çiziniz.

ÖRNEK

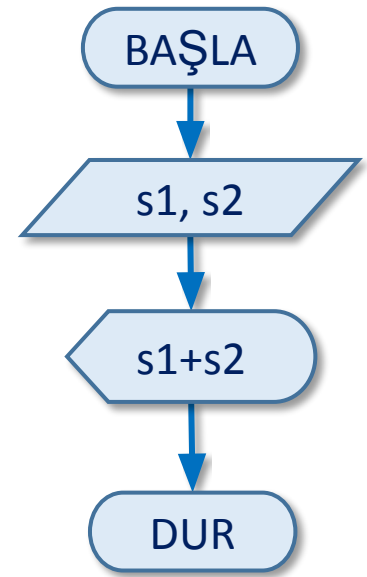
□ Klavyeden girilen iki sayıyı toplayıp sonucunu ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. OKU s1, s2
3. YAZ s1+s2
4. DUR

ÖRNEK

□ Klavyeden girilen iki sayıyı toplayıp sonucunu ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. OKU s1, s2
3. YAZ s1+s2
4. DUR



ÖRNEK

- ▶ Klavyeden girilen x ve y değerlerine göre $x^2 + y$ denkleminin sonucunu hesaplayıp ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çizin.

ÖRNEK

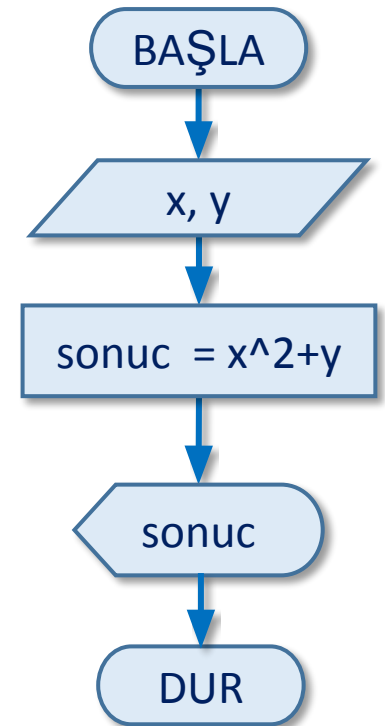
► Klavyeden girilen x ve y değerlerine göre $x^2 + y$ denkleminin sonucunu hesaplayıp ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çizin.

1. BAŞLA
2. OKU x, y
3. $\text{sonuc} = x^2 + y$
4. YAZ sonuc
5. DUR

ÖRNEK

► Klavyeden girilen x ve y değerlerine göre $x^2 + y$ denkleminin sonucunu hesaplayıp ekrana yazdıran yazan programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. OKU x, y
3. $\text{sonuc} = x^2 + y$
4. YAZ sonuc
5. DUR



ÖRNEK

- ▶ Klavyeden girilen x değerlerinin toplamı 100'den büyük oluncaya kadar kullanıcıdan değer alan programın algoritmasını yazınız ve akış diyagramını çizin.

ÖRNEK

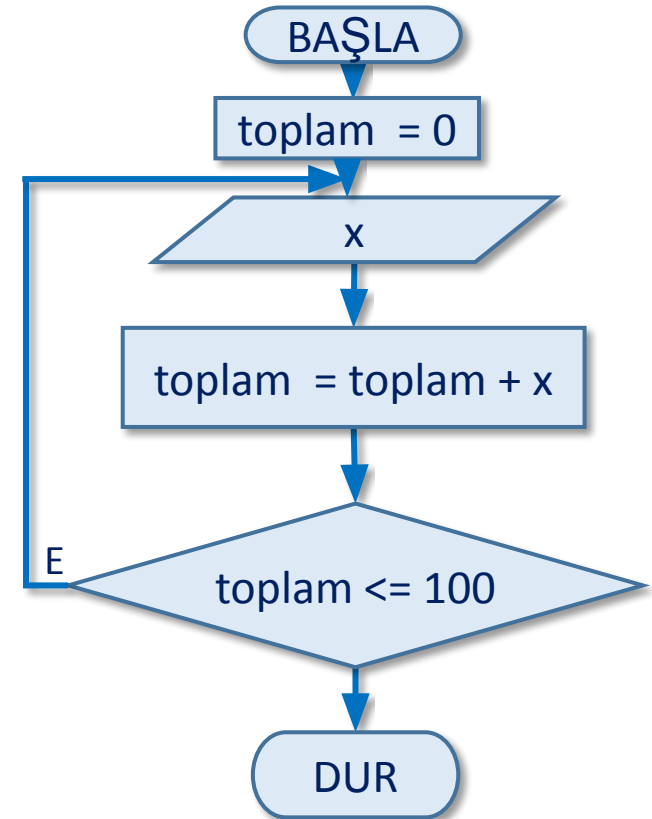
▶ Klavyeden girilen x değerlerinin toplamı 100'den büyük oluncaya kadar kullanıcıdan değer alan programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. toplam = 0
3. OKU x
4. toplam = toplam + x
5. EĞER toplam $\leq$ 100 GİT 3
6. DUR

ÖRNEK

► Klavyeden girilen x değerlerinin toplamı 100'den büyük oluncaya kadar kullanıcıdan değer alan programın algoritmasını yazınız ve akış diyagramını çiziniz.

1. BAŞLA
2. toplam = 0
3. OKU x
4. toplam = toplam + x
5. EĞER toplam $\leq$ 100 GİT 3
6. DUR

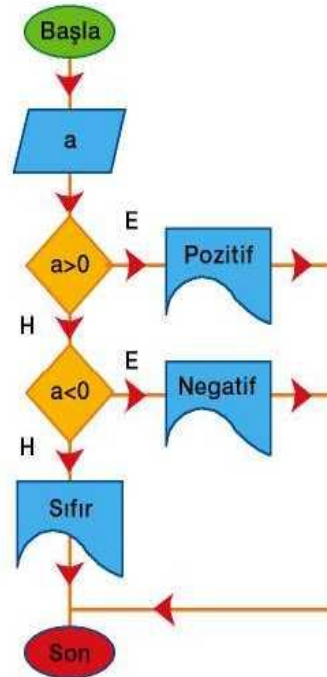


ÖRNEK

- Klavyeden girilen sayının Pozitif - Negatif olma durumunu kontrol eden programın akış diyagramını çiziniz.

ÖRNEK

- Klavyeden girilen sayının Pozitif - Negatif olma durumunu kontrol eden programın akış diyagramını çiziniz.



ÖRNEK

- Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

ÖRNEK

□ Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

□ 1. ÇÖZÜM:

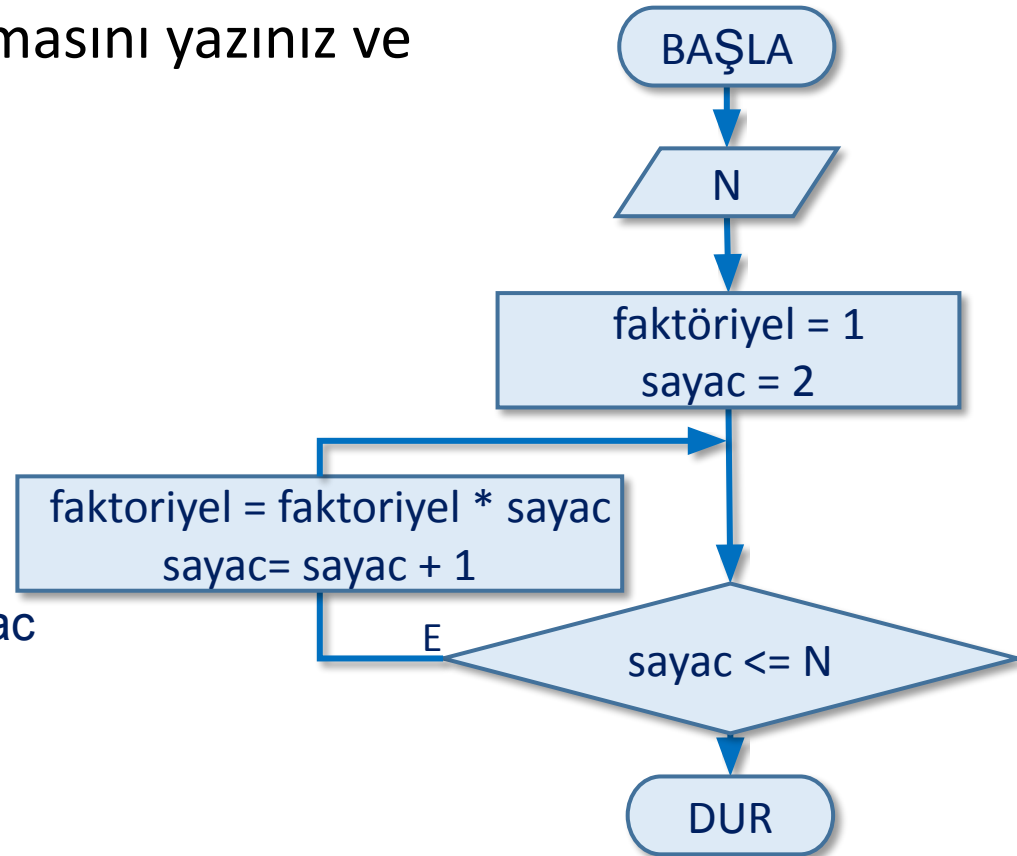
1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. sayac = 2
5. EĞER (sayac $\leq$ N)
 - 5.1. faktoriyel = faktoriyel * sayac
 - 5.2. sayac = sayac + 1
6. YAZ faktoriyel
7. DUR

ÖRNEK

□ Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

□ 1. ÇÖZÜM:

1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. sayac = 2
5. EĞER (sayac $\leq$ N)
 - 5.1. faktoriyel = faktoriyel * sayac
 - 5.2. sayac = sayac + 1
6. YAZ faktoriyel
7. DUR



ÖRNEK

□ Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

□ 2. ÇÖZÜM:

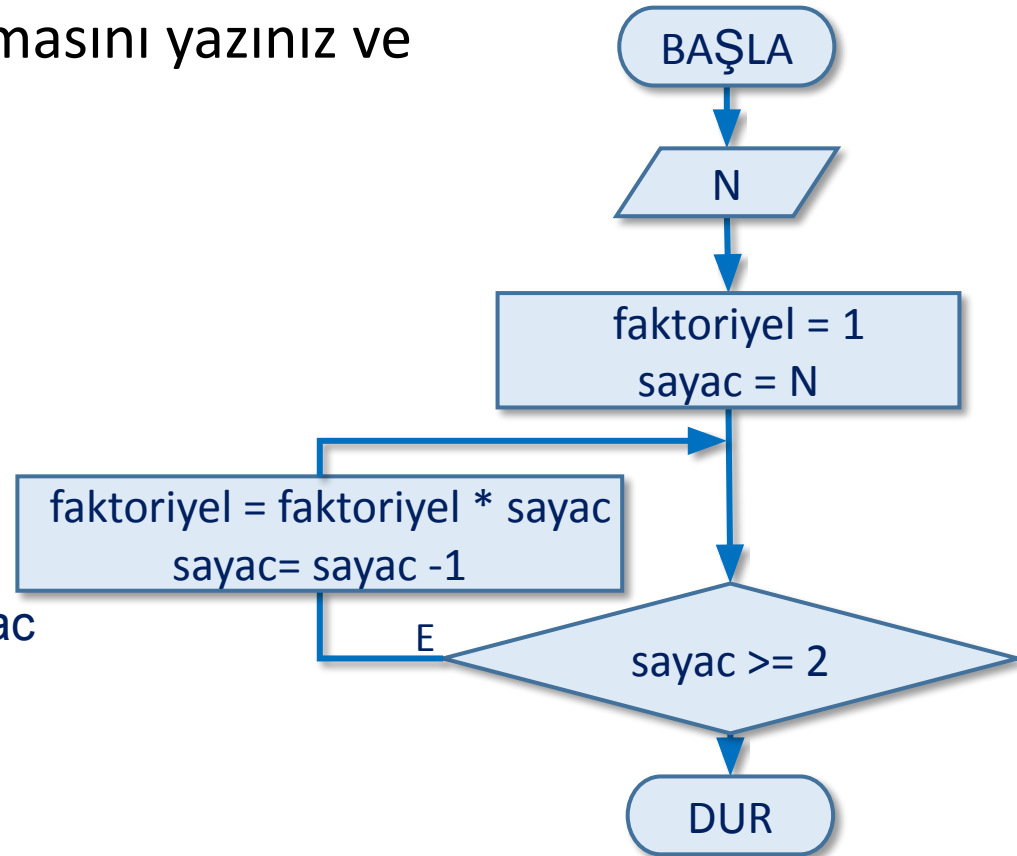
1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. sayac = N
5. EĞER (sayac $\geq$ 2)
 - 5.1. faktoriyel = faktoriyel * sayac
 - 5.2. sayac = sayac - 1
6. YAZ faktoriyel
7. DUR

ÖRNEK

□ Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

□ 2. ÇÖZÜM:

1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. sayac = N
5. EĞER (sayac $\geq$ 2)
 - 5.1. faktoriyel = faktoriyel * sayac
 - 5.2. sayac = sayac - 1
6. YAZ faktoriyel
7. DUR



ÖRNEK

▶ Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

▶ **3. ÇÖZÜM:** $\Rightarrow$ for döngüsü

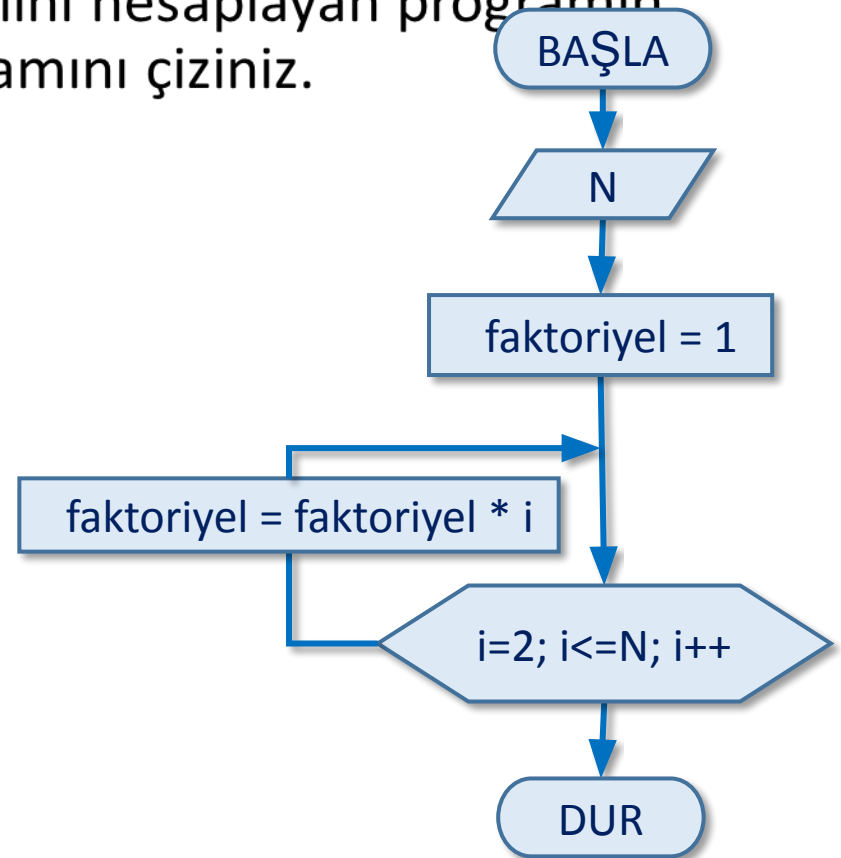
1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. FOR (i=2; i<=N; i++)
 - 4.1. faktoriyel = faktoriyel * i
5. YAZ faktoriyel
6. DUR

ÖRNEK

► Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

► **3. ÇÖZÜM:** $\Rightarrow$ for döngüsü

1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. FOR (i=2; i<=N; i++)
 - 4.1. faktoriyel = faktoriyel * i
5. YAZ faktoriyel
6. DUR



ÖRNEK

▶ Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

▶ **4. ÇÖZÜM:** $\Rightarrow$ for döngüsü

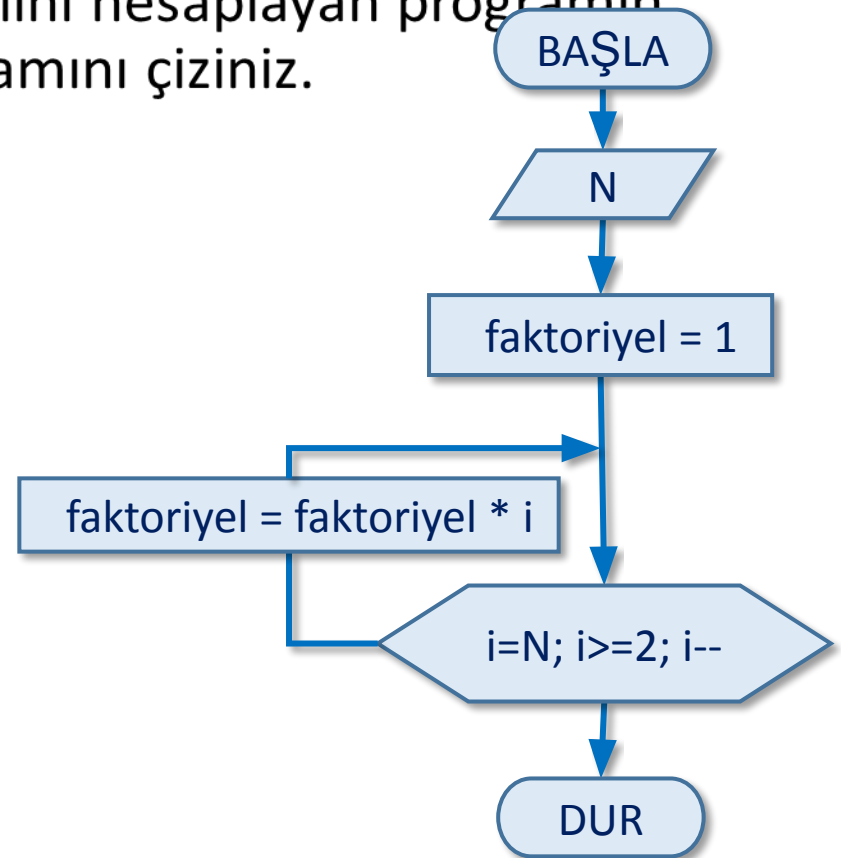
1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. FOR (i=N; i>=2; i--)
 - 4.1. faktoriyel = faktoriyel * i
5. YAZ faktoriyel
6. DUR

ÖRNEK

► Klavyeden girilen sayının faktöriyelini hesaplayan programın algoritmasını yazınız ve akış diyagramını çiziniz.

► **4. ÇÖZÜM:** $\Rightarrow$ for döngüsü

1. BAŞLA
2. OKU N
3. faktoriyel = 1
4. FOR (i=N; i>=2; i--)
 - 4.1. faktoriyel = faktoriyel * i
5. YAZ faktoriyel
6. DUR



Kaynaklar

□ C. Öz, C.Bayılmış, G.Çit «Ders Notları» (Temel İçerikler)

□ Deitel, C++ How To Program, Prentice Hall

□ Prof. Dr. Cemil ÖZ, Programlamaya Giriş Ders Notları

□ Deitel, P., Deitel, H., Java How To Program, Prentice Hall, Ninth Edition, 2012.

□ D.Kılınç «Ders Notları»

□ <https://sites.google.com/site/modernyazilimgelistirme/uml/uml-diyagramlari/use-case-diagramlari>

□ Microsoft Visio Professional 2019

□ Resimler:

□ <https://www.animatedimages.org/img-animated-tea-and-teapot-image-0038-164972.htm>